

Terraform AWS Architecture: End-to-End Build Guide

This project provides a modular, scalable, and secure template for building AWS infrastructure. It is designed to be reusable across Development, Staging, and Production environments.

1. Prerequisites

Before starting, ensure you have:

- An **AWS Account**.
- **Terraform** installed on your laptop ([Get it here](#)).
- **AWS CLI** installed and configured (aws configure in your terminal).
- A **GitHub Account**.

2. Manual Setup (The Foundation)

Step A: AWS Console Setup

1. **Create an S3 Bucket:** Go to the S3 service in AWS, click "Create bucket," and name it something unique (e.g., my-unique-tf-state-123). Enable **Versioning** and **Encryption**.
2. **Create a DynamoDB Table:** Go to DynamoDB, click "Create table."
 - a. **Table Name:** terraform-state-lock
 - b. **Partition Key:** LockID (Type: **String**)
 - c. *Note: These settings ensure your infrastructure state is safe and locked during deployment.*

Step B: Folder Structure

Create the following on your computer:

Plaintext

```
terraform-universal-template/
├── .github/workflows/
│   └── terraform.yml
├── environments/
│   ├── dev/
│   │   └── terraform.tfvars
│   ├── staging/
│   │   └── terraform.tfvars
│   └── prod/
│       └── terraform.tfvars
└── modules/
```

```
| |— networking/ (main.tf, variables.tf, outputs.tf)
| |— security/   (main.tf, variables.tf, outputs.tf)
| |— compute/    (main.tf, variables.tf, outputs.tf)
| — .gitignore
| — backend.tf
| — providers.tf
| — main.tf
| — outputs.tf
| — variables.tf
```

3. Infrastructure Codes

(Copy the files from our previous conversation into these locations. Ensure backend.tf points to your S3 bucket name created in Step A.)

Key Terraform Commands

Run these in your terminal from the terraform-universal-template/ directory:

1. **Initialize:** Prepares your backend and downloads providers.

Bash

```
terraform init
```

2. **Format:** Cleans up your code structure automatically.

Bash

```
terraform fmt -recursive
```

3. **Plan:** Previews what infrastructure will be built.

Bash

```
terraform plan -var-file="environments/dev/terraform.tfvars"
```

4. **Apply:** Builds the infrastructure.

Bash

```
terraform apply -var-file="environments/dev/terraform.tfvars"
```

5. **Destroy:** Deletes the infrastructure (run this to stop costs!).

Bash

```
terraform destroy -var-file="environments/dev/terraform.tfvars"
```

4. GitHub CI/CD Pipeline

Step A: GitHub Secrets

1. Go to your GitHub repo **Settings > Secrets and variables > Actions**.
2. Add two secrets: `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` using your IAM credentials.

Step B: Git Commands

To get your code into GitHub:

Bash

```
git init
```

```
git add .
```

```
git commit -m "Initial commit of Terraform template"
```

```
git branch -M main
```

```
git remote add origin https://github.com/YOUR\_USERNAME/terraform-universal-template.git
```

```
git push -u origin main
```

5. README.md (Copy this into your repo)

Markdown

Terraform AWS Infrastructure Template

This repository contains a modular, production-ready AWS architecture.

Getting Started

1. Create the S3 bucket and DynamoDB lock table in AWS Console.
2. Update ``backend.tf`` with your bucket name.
3. Configure ``AWS_ACCESS_KEY_ID`` and ``AWS_SECRET_ACCESS_KEY`` in GitHub Secrets.
4. Push your code to ``main`` to trigger the CI/CD pipeline.

SSH Access

After the pipeline runs, find your new IP in the GitHub Actions logs.

SSH using your key:

```
`ssh -i "YourKeyName.pem" ec2-user@<IP_ADDRESS>`
```